ChatGPT

Milestone 29: Experiment Registry & Artifact Catalog

Milestone 29 will build on M28's reproducible execution foundation by adding a **persistent experiment registry and artifact catalog**. This milestone implements experiment-tracking features (similar to an MLflow-style registry) that record every run's metadata and output artifacts, and enables searching or querying across past runs. The registry will capture run IDs, configurations, parameters, and key metrics; the catalog will index output artifacts (manifests, metrics, models, etc.) with metadata tags. Together, these features let researchers **find, compare, and reproduce past experiments** without breaking the one-execution-system architecture.

This direction extends M28 naturally: after M28 established *artifact-first, deterministic runs* with safe output roots and uniform entry points [1] [2], the next step is to **track and explore those artifacts systematically**. An experiment registry preserves reproducibility by logging each run's config and results, while an artifact catalog (or metadata index) enables discovery and lineage queries. Both layers sit *above* the core execution engine and use its outputs (manifests, summaries, metrics) as inputs, so **no workflow logic is duplicated or replaced** (in line with the "one execution system, multiple entry points" rule [3]).

Why this is the right next step after M28: M28's safety and integration work makes it easy to run workflows with unique outputs; M29 leverages that by curating those outputs. A registry/catalog adds long-term value: researchers can query past strategy or portfolio runs by parameters or performance, compare experiments, and ensure provenance of artifacts. This aligns with StratLake's reproducible-research goals and adds a **governance/lineage layer** often found in mature data/ML platforms. In short, M28 gave us *reliable runs*; M29 lets us *manage and reuse* their results.

Alternatives Considered

- **Parameter-sweep orchestration:** Extending pipeline definitions to sweep parameters is valuable for research experiments, but this often involves writing new pipeline configs or wrappers. It's lower priority because core pipeline support exists, and ad-hoc scripts can already run repeated experiments. By contrast, a registry is broadly useful across all workflows.
- **Model governance / promotion gates:** While ensuring model quality and documentation (e.g. model cards) is important, much of StratLake's governance (promotion gates, review packs) already exists in M26–M27 [4]. Full-fledged governance (dashboards, manual approval processes) is a heavier production concern and can be deferred. M29 will focus on fundamental artifact tracking rather than policies.
- **Distributed execution hardening:** M28 explicitly avoided distributed locks or DBs (non-goal [5]). Adding a true distributed scheduler or lock service (e.g. Redis, Airflow production setup) would break M28's design constraints and is complex. Instead, we continue using local FS markers for idempotency [2]. Full distributed orchestration can be a future Milestone 30, after laying the registry foundation.

- **Feature store:** A semantic feature catalog or live data ingestion is beyond the current research scope. The engine assumes curated static datasets; adding a feature-store layer would divert from core capabilities.
- **Observability/telemetry:** Internal telemetry (timings, logs) is useful, but can be integrated with the registry/catalyst features. It's lower priority than enabling queryable outputs. Observability can piggyback on the artifact catalog (e.g. adding run durations to the registry) rather than being a separate milestone.
- **Reporting dashboard:** Building a UI on top of artifacts (e.g. web dashboard) could be done after the registry exists; it's noted as a future extension. For M29, we'll provide CLI/notebook query tools and documentation. A full dashboard is beyond scope.

Milestone 29 Issues (Proposed)

1. **Issue: Implement Experiment Registry** (`enhancement`, `milestone-29`, `artifacts`, `tracking`)
   Description: Extend core workflows (CLI or `src.execution` APIs) to append each new run to a registry file (`experiments_registry.jsonl` or SQLite). For *strategy* and other runs, record `run_id`, workflow type (strategy/alpha/portfolio/etc.), config path or parameters, timestamp, and key outputs (e.g. manifest path, metrics JSON path). The registry entry should include enough info for reproduction: config hash, seed, and summary metrics. Use atomic file writes or SQLite transactions to avoid corruption when runs are repeated.
   Context: An early issue envisioned a registry of strategy runs (run ID, params, metrics) [6] [7]. We'll build on that idea using M28's artifact-first outputs.
   Scope/Deliverables: Modify strategy/alpha/portfolio execution code so that on successful completion it writes a registry entry. Provide a CLI command (e.g. `src.cli.run_experiment_registry_update`) that scans completed run directories and updates the registry to a given baseline. Add unit tests verifying that running the same workflow twice does **not** duplicate entries, and that the registry can be queried by run ID or name.
   Acceptance Criteria: New runs automatically get logged in the registry. The registry can be loaded into Python (e.g. via Pandas) and contains all expected fields. Re-running with identical inputs does not duplicate (demonstrated in tests). Each entry has a pointer to the artifact paths (manifest, metrics).

2. **Issue: Extend Registry to Portfolio/Regime Runs** (`enhancement`, `milestone-29`, `registry`)
   Description: Enable the experiment registry to also record *portfolio-level* and *regime-research* executions. Add `run_type = portfolio` or `regime_benchmark` to registry entries. For portfolio runs, include component strategy run IDs (as recorded in M28 manifests) and the allocator/config details. For regime runs (benchmark packs), include the regime taxonomy and variant names. Ensure backward compatibility so that strategy entries are unchanged.
   Context: Similar work was scoped earlier to support portfolio runs in the registry [7]. We now generalize it: all StratLake workflows (strategies, benchmarks, portfolios, etc.) should log to the same registry structure.
   Scope/Deliverables: Update the registry-writing code to accept different workflow types. Update tests to include mock portfolio and regime runs, verifying that entries include the right fields (component runs, regime labels).
   Acceptance Criteria: The registry schema includes a `run_type` field. Portfolio runs appear in the registry with their meta (weights, constraints) and refer to existing strategy run IDs. Regime

benchmark runs appear with config name and comparison IDs. Unit tests confirm entries are created for each workflow.

3. **Issue: Artifact Catalog and Indexing** (`enhancement`, `milestone-29`, `artifacts`)
   Description: Build a catalog of artifacts across runs to support metadata search. For example, create an index (`artifact_catalog.json`) listing each artifact (manifests, model files, equity curves) with metadata like run ID, type, creation date, and tags (strategy name, regime label, etc.). Use existing artifact manifests: parse each run's `manifest.json` and add its key files to the catalog.
   Scope/Deliverables: Provide a script or CLI (e.g. `src.cli.build_artifact_catalog`) that scans all run output directories under the artifacts root and compiles the catalog. The catalog entries should reference the registry (linking run IDs) and include artifact type (`metrics`, `model`, `csv`, etc.). Implement unit tests to verify that catalog entries match files on disk and metadata.
   Acceptance Criteria: Running the catalog builder produces a machine-readable index file. Given a run's directory, its artifacts appear in the catalog with correct metadata. The catalog can be loaded programmatically to find all artifacts of a given type or tag.

4. **Issue: Registry/Artifact Query API** (`enhancement`, `milestone-29`, `cli`, `api`)
   Description: Provide user-friendly access to the registry and catalog. Implement a CLI or notebook API to query/filter experiments (e.g. list runs for strategy "momentum_v1" or runs after date X) and to retrieve artifact info. For example: `python -m src.cli.query_registry --workflow strategy --name momentum_v1 --sort-by metric:sharpe`. Results could be printed as tables or returned as JSON.
   Scope/Deliverables: Add new CLI commands (under `src.cli`, e.g. `run_query_registry`) and corresponding `src.execution` helpers. Support filters on any registry field (workflow, run_date, metrics value thresholds). Write docs/examples illustrating a typical query workflow from a notebook.
   Acceptance Criteria: CLI queries return correct subsets of the registry. For instance, querying by strategy name returns all matching run IDs and metrics. Include tests that simulate a small registry and verify query results.

5. **Issue: Record Lineage and Provenance** (`enhancement`, `milestone-29`, `tracking`)
   Description: Link related runs for provenance. For example, if a portfolio run uses component strategy runs, the registry could record these relationships. If a benchmark pack run produces a regime taxonomy, record the source config of the regime labels. This lets users trace how artifacts depend on earlier runs.
   Scope/Deliverables: Extend registry schema to include parent/child links (e.g. `parent_run_ids`). Automatically populate these from known fields (portfolio manifests list component run IDs, campaign checkpoints list child runs, etc.). Update tests to verify that lineage relationships appear in registry entries.
   Acceptance Criteria: The registry shows that, e.g., a portfolio run's entry contains its component strategy run IDs. Regime run entries include their source taxonomy or policy IDs. Tests cover at least one lineage case.

6. **Issue: Documentation and Examples** (`docs`, `milestone-29`)
   Description: Update documentation to explain the registry and catalog: how entries are logged, how to query them, and how they fit into workflows. Include example code in `docs/examples/notebooks/` demonstrating a complete cycle: run a strategy, run a portfolio, then query the

registry for those runs and load their artifacts.
Scope/Deliverables: New docs in `docs/` (e.g. `docs/experiment_registry.md`). Example notebooks/py scripts. Ensure docs mention that registry features respect M28 principles (explicit output roots, etc.).
Acceptance Criteria: Doc pages and examples can be run (dry-run) without errors and correctly illustrate usage. `run_docs_path_lint` should flag no issues on new docs.

Branch and Implementation Order

- **Branch name:** `feature/m29-experiment-registry` (or similar).
- **Order:** Begin with the **Experiment Registry Core** (Issue 1). Next, **Extend to Portfolio/Regime Runs** (Issue 2), since that builds on the same mechanism. Then create the **Artifact Catalog** (Issue 3) to index run outputs. After that, implement the **Query API** (Issue 4) using the populated registry/catalog. Finally add **Lineage Links** (Issue 5) to tie runs together, and polish with **Docs/Examples** (Issue 6). This order ensures each component has its inputs available (e.g. queries need the registry).

Risks and Non-Goals

- **Concurrency on the registry file:** As with output roots in M28, multiple runs writing to the registry simultaneously could collide. We will adopt the same local FS safeguards (atomic writes, marker files, and unique output roots) [2]. By default each run writes to its own root, so appending to a centralized registry can be done after runs complete (e.g. via a CLI or at program exit). This avoids introducing a new distributed locking service (which M28 explicitly avoided [5]).
- **Scope creep:** We will *not* implement a full database-backed tracking system or UI. The registry will be file-based (JSON/CSV/SQLite) and designed for research use, not enterprise production (consistent with M28's non-goals of avoiding heavy infrastructure [5]). The catalog will be simple (no fancy ontology) and rely on existing metadata.
- **One-execution-system principle:** All new code must **not duplicate workflow logic**. The registry and catalog are meta-systems that sit *around* StratLake's execution surfaces. We must not change how strategies or pipelines work; we only hook into their completion and read their outputs. This preserves the M28 architecture rule [3].
- **Volume of data:** If hundreds of runs are recorded, the registry file could grow large. We assume medium-scale use (hundreds, not millions, of runs). If needed, future milestones could introduce partitioning or databases, but M29 will target local-file or SQLite solutions only.
- **Non-goals:** M29 will not address live trading, real-time data ingestion, or require external schedulers. It remains a research-layer feature. We also do not undertake schema evolution support or feature-store integration here; those are separate agenda items.

Preserving M28 Architecture

Every M29 feature is built on *existing artifacts and APIs*. We will **not introduce new execution frameworks or duplication**. For example, registry entries are created by calling the same `src.execution` functions or CLI commands that M28 used; we merely extend those calls to log metadata. The artifact catalog reads the `manifest.json` and other stable outputs that M28 workflows already produce. In other words, **we extend, not replace, the canonical outputs** [1] [2]. All M28 safeguards (unique output roots, atomic writes, markers) still apply to the runs themselves, and we reuse `src/artifacts/safety.py` helpers if

needed for writing the registry file. Thus, M29 supplements M28 without violating its design: the core engine and output contracts remain the single source of truth.

Resume/Interview Framing

Milestone 29 can be framed as a major step toward an end-to-end reproducible research platform. On a resume or in an interview, one might say:

- *"In Milestone 29, I built an experiment-tracking registry and artifact catalog on top of our StratLake engine. This added metadata logging (config, metrics, run IDs) and a query interface, enabling searchable experiment histories and lineage without altering the core execution logic. By leveraging M28's artifact-first design, I ensured all new features used existing manifests and summaries, preserving one execution system across CLI, API, notebooks, and orchestrators [3] [2]. The result is a data/ML engineering–style experiment platform (akin to MLflow), which demonstrates mature handling of reproducibility, provenance, and searchable metadata in a quant research context."*

This highlights data platform maturity (experiment tracking, metadata indexing) and ties directly back to M28 achievements.

---

[1] [3] m28_release_notes.md
https://github.com/christophermoverton/stratlake-trade-engine/blob/aa211605d0af5257beb469ed5d467055066168ea/docs/m28_release_notes.md

[2] [5] concurrency_and_idempotency.md
https://github.com/christophermoverton/stratlake-trade-engine/blob/aa211605d0af5257beb469ed5d467055066168ea/docs/concurrency_and_idempotency.md

[4] milestone_28_unified_regime_research_case_study.md
https://github.com/christophermoverton/stratlake-trade-engine/blob/aa211605d0af5257beb469ed5d467055066168ea/docs/milestone_28_unified_regime_research_case_study.md

[6] Experiment Registry
https://github.com/christophermoverton/stratlake-trade-engine/issues/32

[7] Portfolio Registry Integration
https://github.com/christophermoverton/stratlake-trade-engine/issues/86